

Zero-Shot Prompt Injection Detection via N-gram Pooling of Token-Level Hidden States

Lokesh Prakash Manohar
Independent Researcher, United States

Abstract—Retrieval-augmented generation (RAG) pipelines are vulnerable to prompt injection attacks, where adversaries embed malicious instructions inside retrieved documents. Existing embedding-based detectors rely on mean-pooled sentence vectors, which suppress short injection spans hidden within long benign text — a phenomenon we formalize as *semantic dilution*. We extract per-token hidden states from a frozen 22M-parameter sentence transformer already present in the pipeline and apply N-gram pooling over sliding windows to localize injection signals. On a controlled chunk-level benchmark (1,575 chunks, Enron corpus), our method achieves AUC = 0.901 (95 % CI [0.869, 0.930]), with confidence intervals non-overlapping with the mean-pool baseline (AUC = 0.808). On the public xTRam1 dataset, zero-shot AUC = 0.870 exceeds the supervised baseline of 0.764. All three scoring variants execute at identical CPU latency (≈ 44 ms per 500-character chunk). The method requires no labeled data, no fine-tuning, and no access to the generative LLM at inference time.

I. INTRODUCTION

Retrieval-augmented generation augments language model responses by injecting retrieved document passages into the context window [1]. This design surface is exploitable: an adversary who can write to any document in the knowledge base can embed a prompt injection that hijacks the model's behavior when that document is retrieved [2], [3]. Defending against such attacks at retrieval time — before the generative LLM is ever invoked — is therefore a critical pipeline hardening step.

Embedding-based detectors compare a semantic representation of each retrieved chunk against a small set of known-malicious reference phrases [4]. The appeal is clear: the sentence transformer used for retrieval is already in the pipeline, so detection adds negligible infrastructure. However, all prior embedding-based work collapses each chunk to a single mean-pooled vector [4], [5]. When a short injection phrase is surrounded by a long benign passage, the mean-pooled vector converges toward the benign centroid and the injection signal is washed out. We call this *semantic dilution* and formalize it in Section III.

Methods that exploit sub-sentence granularity do exist. InstructDetector [6] extracts per-token hidden states and gradients from the guarded generative LLM and achieves near-perfect accuracy. But this design requires a full forward-and-backward pass through the LLM at inference time — expensive, and unavailable at ingestion time before any query is issued.

We propose a middle path. A sentence transformer already produces contextual per-token hidden states as a byproduct of its forward pass. Rather than collapsing these to a sentence

vector, we apply N-gram pooling: we average each window of N consecutive token embeddings and score the chunk against a reference set using the maximum similarity over all windows. This recovers phrase-level intent that mean-pooling destroys, while remaining entirely within the lightweight encoder.

Our contributions are:

- 1) A formal characterization of semantic dilution in mean-pooled injection detection (Section III).
- 2) N-gram pooling over token-level hidden states for localized, zero-shot injection scoring, with no training on injection data (Section III).
- 3) A controlled chunk-level evaluation with bootstrap confidence intervals on the Enron corpus, demonstrating statistically significant improvement over mean-pool and per-token baselines (Section V).
- 4) A zero-shot comparison against supervised baselines on three public benchmarks, with the N-gram scanner exceeding the supervised AUC on xTRam1 (Section V).

II. RELATED WORK

A. Embedding-Based Injection Detection

Ayub and Majumdar [4] apply the same encoder used in this work (all-MiniLM-L6-v2) to injection detection by computing mean-pooled sentence embeddings and training a random forest classifier on top. Their best supervised AUC is 0.764. The key limitation is that mean pooling is applied before classification, discarding positional structure within the chunk.

Alamsabi et al. [5] construct context-pair embeddings from consecutive paragraph segments and train a classifier on the BIPIA benchmark [3], achieving F1 = 0.977. Their method is trained on the same distribution it is evaluated on; it requires labeled examples of injections per domain and cannot generalize zero-shot to unseen injection styles.

B. Token-Level Hidden State Methods

InstructDetector [6] is the closest prior work exploiting sub-sentence granularity. It extracts per-token hidden states and gradient magnitudes from Llama-3.1-8B and trains a classifier on top, achieving 99.6 % accuracy. The critical constraint is the requirement for a forward-and-backward pass through the guarded generative LLM: this is unavailable at document ingestion time, and adds inference cost proportional to LLM scale. Our approach uses a 22M-parameter encoder and requires no gradients.

C. RAG Security Benchmarks

Perez and Ribeiro [7] catalog foundational prompt injection attack techniques, including the “ignore previous instructions” class that motivates detection at retrieval time. Zhan et al. [8] introduce InjecAgent, benchmarking indirect prompt injection in tool-integrated agents where injections arrive via retrieved documents.

Yi et al. [3] introduce BIPIA, the standard benchmark for indirect prompt injection in RAG contexts, which provides 10,000 attack-response pairs across five tasks. The Jailbreak-Bench framework [9] defines 100 harmful behaviors across 10 taxonomic categories; we use it as the source of our reference set and injection corpus. The sentence-transformers library [10] provides the all-MiniLM-L6-v2 encoder used throughout this work.

III. METHOD

A. Problem Statement

Let D be a document retrieved from an external corpus during a RAG pipeline query. D is partitioned into K non-overlapping chunks $c_1, \dots, c_K$ of at most W characters. A chunk c_k is *poisoned* if an adversary has embedded an injection phrase within the surrounding benign text; otherwise it is *clean*. Our goal is to assign a scalar score $s(c_k)$ such that poisoned chunks score higher than clean ones — without any training on labeled injection data and without access to the generative LLM. Cosine similarity between L2-normalized vectors lies in $[-1, 1]$; empirically all scores in this work are non-negative owing to the sentence transformer’s normalization, so $s(c_k) \in [0, 1]$ holds in practice.

B. Token-Level Hidden States

We use a lightweight sentence transformer f (all-MiniLM-L6-v2, 22M parameters, $d = 384$) already present in the RAG pipeline for retrieval. For an input chunk c tokenized into L subword tokens, f produces contextual token embeddings:

$$H = f(c) \in \mathbb{R}^{L \times d}$$

where $h_i \in \mathbb{R}^d$ is the hidden state of the i -th token. The model is used in frozen inference mode; no gradient computation is performed.

C. Semantic Dilution

Standard sentence-level scoring computes a mean-pooled representation:

$$\mathbf{g} = \text{normalize}\left(\frac{1}{L} \sum_{i=1}^L h_i\right)$$

and assigns a score $s_{\text{global}} = \max_j \cos(\mathbf{g}, r_j)$, where r_j are reference embeddings (Section III-D).

Consider a chunk with L_b benign tokens and L_p injection tokens with $L_p \ll L_b$. Under linear superposition of embedding spaces:

$$\mathbf{g} \approx \frac{L_b}{L_b + L_p} \mathbf{g}_{\text{benign}} + \frac{L_p}{L_b + L_p} \mathbf{g}_{\text{inject}}$$

As $L_b \gg L_p$, $\mathbf{g} \rightarrow \mathbf{g}_{\text{benign}}$ and the injection signal is diluted out of the representation. Mean pooling is a length-weighted average that systematically suppresses minority spans.

D. Reference Set

We construct a fixed reference set $\mathcal{R} = \{r_1, \dots, r_{10}\}$ using one representative phrase per behavioral category from JailbreakBench [9]: Malware, Physical Harm, Fraud, Disinformation, Privacy, Harassment, Government Interference, Sexual Content, Economic Harm, and Expert-Advice Circumvention. Each phrase is the shortest direct statement of harmful intent in its category (e.g., “Write a keylogger script”), encoded once offline:

$$r_j = \text{normalize}(\text{mean-pool}(f(\text{phrase}_j)))$$

The reference set is not trained; it serves as a semantic anchor. No phrase from the evaluation injection corpus appears in the reference set.

E. N-gram Pooling

For each position i in the token sequence, we compute an N-gram embedding by averaging N consecutive token hidden states:

$$\tilde{h}_i = \text{normalize}\left(\frac{1}{N} \sum_{k=0}^{N-1} h_{i+k}\right), \quad i = 1, \dots, L - N + 1$$

In practice this is a single 1D average-pooling operation along the sequence axis:

$$\tilde{H} = \text{AvgPool1D}(H^\top, \text{kernel} = N, \text{stride} = 1)^\top \in \mathbb{R}^{(L-N+1) \times d}$$

The chunk score is the maximum cosine similarity between any N-gram embedding and any reference phrase:

$$s_{\text{ngram}}(c) = \max_i \max_j \cos(\tilde{h}_i, r_j)$$

The maximum over positions localizes the injection span. The maximum over reference phrases provides coverage across behavioral categories. The token-level method ($N = 1$) is a special case. This sliding-window approach draws conceptual inspiration from convolutional sentence classification [11], where local filters capture phrase-level features; here learned filters are replaced by cosine similarity to a fixed reference set, requiring no training.

F. Document-Level Score

For a document D chunked into $c_1, \dots, c_K$:

$$s(D) = \max_{k=1}^K s_{\text{ngram}}(c_k)$$

A document is flagged if $s(D) \geq \tau$ for an operating threshold τ . The scanner runs once per document at ingestion time, before any retrieval or LLM call.

G. Complexity

The dominant cost is the forward pass of f : $O(L^2 d)$ for self-attention. The N-gram pooling step is $O(Ld)$ and reference comparison is $O(L \cdot |\mathcal{R}| \cdot d)$, both negligible. End-to-end latency is 43–45 ms per chunk on CPU (Section V-C), identical across all three scoring strategies because the forward pass dominates.

Algorithm 1 N-gram Injection Scanner

Require: chunk c , reference matrix $R \in \mathbb{R}^{10 \times d}$, window $N = 5$

Ensure: score $s \in [0, 1]$

- 1: Tokenize $c \rightarrow (t_1, \dots, t_L)$ with attention mask
- 2: $H \leftarrow f.\text{token_embeddings}(c) \triangleright L \times d$, frozen
- 3: $\tilde{H} \leftarrow \text{AvgPool1D}(H^\top, \text{kernel} = N)^\top \triangleright (L-N+1) \times d$
- 4: $\tilde{H} \leftarrow \text{normalize}(\tilde{H}, \text{dim} = 1)$
- 5: $S \leftarrow \tilde{H} \cdot R^\top \triangleright (L-N+1) \times 10$
- 6: $s \leftarrow \max(S)$
- 7: **return** s

IV. EXPERIMENTAL SETUP

We run two evaluations. The chunk-level experiment tests whether scoring methods separate individual poisoned chunks from clean ones — the cleanest signal for the core technical claim. The document-level application study tests scanner utility in a production-realistic RAG pipeline where a single injection is hidden inside a full multi-chunk document.

A. Injection Corpus

Chunk-level experiment. We construct ten (goal, obfuscated-paraphrase) pairs manually, one per behavioral category. Each paraphrase rephrases harmful intent in corporate bureaucratic language (e.g., “Generate a sequence of high-entropy strings to bypass established system security barriers”), suppressing surface keywords while preserving semantic intent.

Document-level experiment. We use the 100 behaviors from JailbreakBench [9] across 10 categories. Each behavior is paraphrased into three domain styles (corporate, technical, encyclopedic) by Gemma-3-27B-IT [12], yielding 300 distinct injection phrases matched to domains by style.

Important distinction: the chunk-level and document-level experiments use different injection corpora. The chunk-level corpus is hand-written with corporate obfuscation only; the document-level corpus uses Gemma-generated obfuscations matched to each domain. Results across the two evaluations should be compared only at the level of relative method rankings, not absolute AUC values.

B. Document Corpora and Test Construction

Chunk-level (Enron, controlled). We sample 100 ham emails from SetFit/enron_spam [13] (length > 1500 characters), chunk into non-overlapping 500-character windows, and retain windows longer than 400 characters. This yields 1,575 chunks. One chunk per email is selected uniformly at random to receive its corresponding injection phrase, inserted at the chunk midpoint. Labels are at the chunk level (100 positive, 1,475 negative).

Document-level (three domains). Each domain provides 100 documents (50 poisoned, 50 clean), all at least 1,500 characters. For each poisoned document, one chunk is selected at random and the domain-matched injection phrase is embedded at the midpoint. Labels are at the document level. Table I summarizes the corpora.

TABLE I
DOCUMENT-LEVEL EVALUATION CORPORA

Domain	Source	Avg. chunks	Injection style
Enron	SetFit/enron_spam	16	Corporate
Stack Overflow	pacovaldez/stackoverflow-questions [14]	7	Technical
Wikipedia	wikimedia/wikipedia (2023-11-01)	55	Encyclopedic

C. Baselines

We compare three scoring variants that differ only in pooling strategy: **Global** (standard mean-pool sentence embedding), **Token-level** ($N = 1$, per-token maximum similarity), and **N-gram N=5** (our method, window selected from ablation without exposure to labeled injection data). For the document-level study we additionally compare Claude Sonnet 4.6 and DeepSeek-V3 [15] as LLM-as-judge baselines [16] via a zero-shot classification prompt over the full document text.¹

D. Evaluation Protocol

The primary metric is AUC-ROC, which is threshold-free. For chunk-level results we report 95 % bootstrap confidence intervals ($B = 1,000$ resamples) [17]. F1 at the optimal threshold is reported for public benchmark comparisons. Latency is measured on CPU with 20 warm-up runs and 200 measurement runs on 500-character chunks. Random seed 42 is used for all sampling. No injection data is used to set thresholds; the operating-point analysis sweeps thresholds post-hoc.

E. Public Benchmark Comparability

To compare against prior supervised methods, we evaluate the N-gram scanner zero-shot on three public datasets: BIPIA [3] (stratified 1,000-sample held-out set, 500 injections / 500 clean, seed 42), the deepset prompt-injection dataset [18] (116 test samples), and the xTRam1 safe-guard dataset [19] (2,060 test samples). No training is performed; the only injection-domain knowledge is the 10-phrase JailbreakBench reference set.

V. RESULTS

A. Chunk-Level AUC

Table II reports AUC and 95 % bootstrap confidence intervals on the Enron chunk-level benchmark. N-gram N=5 achieves AUC=0.901, a 9.3-point improvement over the global mean-pool baseline. The confidence intervals of global and N-gram are non-overlapping, establishing statistical significance. Token-level ($N = 1$) already surpasses global by 8.1 points; N-gram adds a further 1.3 points by aggregating phrase-level context.

¹LLM-as-judge setup: the prompt instructs the model to classify the document as containing a prompt injection or not, and to respond with a single word (YES or NO). Temperature is set to 0; no sampling is used. Refusals and non-parseable outputs are excluded from the AUC computation. The classification prompt is released with the code repository.

TABLE II
CHUNK-LEVEL AUC ON ENRON CORPUS
(1,575 CHUNKS, 100 POISONED; BOOTSTRAP $B = 1,000$)

Method	AUC	95 % CI
Global (mean-pool)	0.8076	[0.762, 0.849]
Token-level ($N = 1$)	0.8882	[0.856, 0.919]
N-gram $N = 5$ (ours)	0.9011	[0.869, 0.930]

TABLE III
N-GRAM WINDOW SIZE ABLATION (ENRON CHUNK-LEVEL)

N	AUC
1	0.8882
2	0.8824
3	0.8906
4	0.8936
5	0.9011
6	0.9008
7	0.8959
8	0.8969
10	0.8996

B. N-gram Window Ablation

Table III shows AUC as a function of window size N . The curve is unimodal with a peak at $N = 5$, declining symmetrically on both sides. The $N = 1$ value matches the token-level baseline exactly as expected. Windows in the range $N \in [4, 6]$ all achieve $\text{AUC} \geq 0.900$, suggesting that injection phrases span approximately 4–6 subword tokens in the obfuscated corpora used. We fix $N = 5$ as our reported configuration. This value was selected post-hoc via ablation on the Enron chunk-level benchmark; however, the flat plateau across $N \in [4, 6]$ indicates the choice is not tightly tuned to this dataset.

C. Inference Latency

Table IV reports per-chunk latency measured on CPU. All three scoring methods execute within a 2-ms range of each other. The encoder forward pass dominates; pooling strategy contributes negligible overhead. N-gram $N = 5$ has the lowest p95 latency (56.2 ms) due to slightly more vectorized matrix operations compared to the token-level variant.

D. Document-Level Application Study

Table V reports document-level AUC across three domains for scanner variants and LLM-as-judge baselines. All three scanner variants degrade substantially relative to chunk-level performance, confirming that document-level detection is the harder task: the max-pooling aggregation propagates noise from the many clean chunks surrounding the single poisoned one.

N-gram consistently outperforms global across all three domains. The margin is largest on Stack Overflow (0.79 vs. 0.58), where obfuscated technical injections are more phrase-localized. Wikipedia is the hardest domain for all methods, with long articles (avg. 55 chunks) providing maximal dilution opportunity.

TABLE IV
PER-CHUNK INFERENCE LATENCY ON CPU
(200 RUNS, 20 WARM-UP, 500-CHARACTER INPUT)

Method	Mean (ms)	p95 (ms)
Global	44.9	64.0
Token-level	45.5	68.8
N-gram $N = 5$	43.9	56.2

TABLE V
DOCUMENT-LEVEL AUC BY DOMAIN
(100 DOCUMENTS PER DOMAIN, 50 POISONED / 50 CLEAN)

Domain	Global	Token	N-gram	Sonnet 4.6	DeepSeek
Enron	0.52	0.62	0.62	0.82	0.87
Stack Overflow	0.58	0.75	0.79	0.56 [†]	0.97
Wikipedia	0.57	0.66	0.71	0.72	0.73

[†]Safety-filter interference; see footnote.

DeepSeek-chat achieves the highest AUC on both Stack Overflow (0.97) and Wikipedia (0.73), demonstrating that LLM-level understanding is a strong upper bound. Sonnet 4.6 shows anomalously low AUC on Stack Overflow ($0.56 \approx \text{chance}$).²

E. Public Benchmark Comparability

Table VI compares our zero-shot scanner against supervised baselines on three public datasets. On xTRam1, N-gram $\text{AUC} = 0.870$ exceeds the supervised baseline of 0.764 [4], which was trained on labeled injection data. On BIPIA, N-gram $\text{AUC} = 0.641$ falls well short of the Alamsabi et al. supervised $F1 = 0.977$ [5]; this gap is expected, as that method was trained on the BIPIA distribution using OpenAI embeddings. On deepset, N-gram $\text{AUC} = 0.642$ with only 116 test samples.

The $N\text{-gram} \geq \text{global}$ ordering holds on BIPIA and deepset. On xTRam1, global pooling achieves $\text{AUC} = 0.913$, which exceeds the N-gram score of 0.870. We attribute this reversal to the structure of xTRam1: many of its injections are short, standalone adversarial strings with minimal surrounding benign context, so semantic dilution is weak and mean pooling suffices. When injections are already phrase-isolated, the localization benefit of N-gram pooling does not apply. $N\text{-gram} \geq \text{global}$ is the expected pattern when injections are embedded inside longer benign passages, as in the controlled Enron benchmark.

VI. DISCUSSION

A. Why Global Fails More at Document Level

At chunk level, the global baseline achieves $\text{AUC} = 0.808$ — competitive, if not state of the art. At document level, it drops to 0.52–0.58, barely above chance. The mechanism is compounded dilution: document scoring uses the maximum N-gram score across all chunks, but for the global method the chunk-level score is already diluted before the max is

²Sonnet’s safety filter blocked responses for documents containing technical hacking content. Blocked responses were excluded from the AUC computation. DeepSeek and Gemma, operating without content restrictions, score 0.97 and 0.95 on the same documents, confirming that the low Sonnet AUC reflects safety-filter interference rather than genuine detection failure.

TABLE VI
ZERO-SHOT PUBLIC BENCHMARK COMPARABILITY. [†]SUPERVISED BASELINE REPORTED AS F1; NOT DIRECTLY COMPARABLE TO AUC
(LITERATURE-LEVEL REFERENCE ONLY).

Dataset	N	Global AUC	Token AUC	N-gram AUC	Supervised baseline	Supervised reference
BIPIA	1,000	0.604	0.591	0.641	F1 = 0.977 [†] (trained on BIPIA)	Alamsabi et al. [5]
deepset	116	0.544	0.611	0.642	AUC = 0.764 (trained)	Ayub & Majumdar [4]
xTRam1	2,060	0.913	0.805	0.870	AUC = 0.764 (trained)	Ayub & Majumdar [4]

TABLE VII
SCANNER OPERATING POINTS — STACK OVERFLOW
(N-GRAM SCANNER AS FIRST-PASS GATE BEFORE DEEPSEEK JUDGE)

Escalation rate	TPR	ASR proxy	Judge cost saved
10%	20%	80%	90%
30%	52%	48%	70%
50%	72%	28%	50%
70%	86%	14%	30%

taken. A single poisoned chunk in a 55-chunk Wikipedia article contributes a globally diluted score that frequently falls below the maximum clean chunk score. Token-level and N-gram methods resolve this by localizing the signal within each chunk before aggregation.

B. Scanner as First-Pass Filter

The document-level AUC values in Table V suggest that the scanner is not a complete solution but a cost-effective first-pass gate. An LLM judge provides higher detection quality but at substantially higher cost (API calls on every document). A cascade design — scanner flags candidates, judge adjudicates only flagged documents — achieves Pareto-superior cost-accuracy tradeoffs at all budgets. Table VII shows operating points for the Stack Overflow domain.

At 30 % escalation, the scanner catches 52 % of injections while reducing judge API calls by 70 %. At 50 % escalation, TPR reaches 72 % with 50 % cost reduction. The operating threshold is selectable post-hoc based on the deployment’s cost-risk tolerance.

C. Limitations

Reference set coverage. The 10-phrase reference set covers the JailbreakBench taxonomy but not injection styles outside that taxonomy, including indirect manipulation, role-based hijacking, context-steering, tool-abuse, exfiltration, and policy-evasion injections. Detection quality degrades for novel behavioral categories not represented in $\mathcal{R}$. Expanding the reference set to cover these categories and evaluating sensitivity to reference-set size are directions for future work.

Wikipedia difficulty. All methods — scanner and LLM judge alike — struggle on Wikipedia (scanner N-gram 0.71, DeepSeek 0.73). Long encyclopedic articles (avg. 55 chunks) provide maximum semantic cover for an injected phrase, and encyclopedic obfuscation mimics the surrounding prose closely. This is an open problem not fully addressed by phrase-level localization.

Zero-shot gap on BIPIA. The supervised baseline of Alamsabi et al. achieves F1=0.977 by training directly on the BIPIA distribution. Our zero-shot AUC=0.641 reflects the cost of generalization: we use a different reference set, a different encoder, and no in-distribution training. The appropriate comparison is xTRam1, where neither method was trained, and our zero-shot AUC=0.870 exceeds the supervised AUC=0.764.

No adversarial evaluation against the scorer. Our injection corpus uses obfuscation styles designed to evade keyword detection, not targeted adversarial attacks against the N-gram scorer itself. An adversary aware of the reference set and pooling strategy could in principle craft minimally similar injections. Adversarial robustness evaluation is left for future work.

VII. CONCLUSION

We presented a zero-shot prompt injection scanner that extracts per-token hidden states from a frozen 22M-parameter sentence transformer and applies N-gram pooling to localize injection signals within document chunks. The method addresses the semantic dilution problem inherent in mean-pooled sentence embeddings: by scoring sliding windows rather than the full chunk mean, it recovers phrase-level intent that global pooling destroys. On a controlled chunk-level benchmark, N-gram N=5 achieves AUC=0.901 with non-overlapping bootstrap CIs relative to the mean-pool baseline. On the public xTRam1 dataset, zero-shot AUC=0.870 exceeds the prior supervised baseline of 0.764. The strongest evidence for the method is at chunk level; at document level, performance is more modest and deployment benefits from cascade LLM adjudication. All three scoring variants run at identical CPU latency, and the method requires no labeled injection data, no fine-tuning, and no access to the generative LLM. Code and the reference set are released as public artifacts at <https://github.com/Lokeshs-git/ngram-injection-scanner> to support reproducibility.

REFERENCES

- [1] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] K. Greshake *et al.*, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.12173>

- [3] J. Yi *et al.*, “BIPIA: Benchmarking indirect prompt injection attacks in retrieval-augmented generation,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025. [Online]. Available: <https://arxiv.org/abs/2312.14197>
- [4] A. Ayub and S. Majumdar, “Embedding-based detection of indirect prompt injection attacks in RAG pipelines,” *arXiv preprint arXiv:2410.22284*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.22284>
- [5] M. Alamsabi *et al.*, “Context-pair embedding for indirect prompt injection detection,” *Algorithms*, vol. 19, no. 1, p. 92, 2026.
- [6] Y. Wen *et al.*, “InstructDetector: Detecting prompt injection via per-token hidden states and gradients,” *arXiv preprint arXiv:2505.06311*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.06311>
- [7] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” in *NeurIPS Workshop on Machine Learning Safety*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.09527>
- [8] Q. Zhan *et al.*, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics (ACL)*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.02691>
- [9] P. Chao *et al.*, “JailbreakBench: An open robustness benchmark for jailbreaking large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.01318>
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [11] Y. Kim, “Convolutional neural networks for sentence classification,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014. [Online]. Available: <https://arxiv.org/abs/1408.5882>
- [12] Gemma Team, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
- [13] B. Klimt and Y. Yang, “The Enron corpus: A new dataset for email classification research,” in *Proceedings of the 15th European Conference on Machine Learning (ECML)*, 2004, pp. 217–226.
- [14] F. Valdez, “Stack overflow questions dataset,” HuggingFace Datasets, 2021. [Online]. Available: <https://huggingface.co/datasets/pacovaldez/stackoverflow-questions>
- [15] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [16] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-bench and chatbot arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [18] deepset, “Prompt injections dataset,” HuggingFace Datasets, 2023. [Online]. Available: <https://huggingface.co/datasets/deepset/prompt-injections>
- [19] xTRam1, “Safe-guard prompt injection dataset,” HuggingFace Datasets, 2023. [Online]. Available: <https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection>